Company and Product Overview

Product, users, capabilities, maturity and repository evidence.

Summary

This document explains the subject in straightforward professional English, then adds the engineering detail needed to build, operate, troubleshoot or review it. Claims are based on the repository evidence snapshot; missing source details are called out instead of guessed.

Document details

Edition 2.0 | Evidence date: 14 August 2026 | Repository: rmorampudi09-arch/Craves-Build-platform | Product evidence snapshot: main @ 3225f7fa531a6b07185fb5e034f7930f8bd8b571

Summary and how to use this document

Product, users, capabilities, maturity and repository evidence. The purpose is to make the system understandable to a new team member while still giving engineers enough detail to trace ownership, data flow, deployment impact and failure handling.

Current implementation

Direct code, README, migration, pipeline or commit evidence exists on the reviewed main snapshot.

Foundation / partial

Useful groundwork exists, but the repository does not prove a complete production runtime for every part.

Legacy / reference

Older implementation is retained for history or compatibility and is not treated as the preferred runtime.

Needs source confirmation

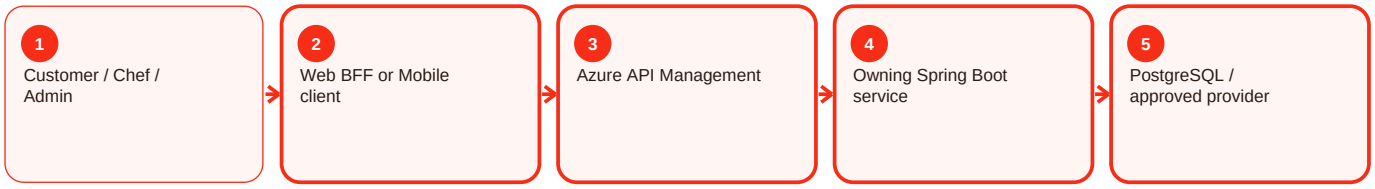
A referenced artifact or exact value could not be verified and is therefore not invented.

Reading order

- Start with the summary to understand the responsibility in normal language.
- Use process diagrams to follow requests across client, APIM, services, data and providers.
- Use the troubleshooting sections to diagnose by evidence rather than by retrying blindly.
- Use deployment and validation sections before or after changing a component.
- Use evidence status to separate proven behavior from gaps and future work.

How the request moves through Craves

The exact components depend on the feature, but the normal pattern is consistent. The user interface starts the request, the gateway and owning service enforce trust, the service writes authoritative state, and provider integrations remain behind controlled boundaries.



Key rule

A screen can hide or show an action for usability, but that is not security. APIM and the owning backend still validate the caller, role, ownership and current business state. Likewise, provider success is not accepted as Craves state until the backend verifies and records the result.

Marketplace actors - Overview and responsibility

Current implementation

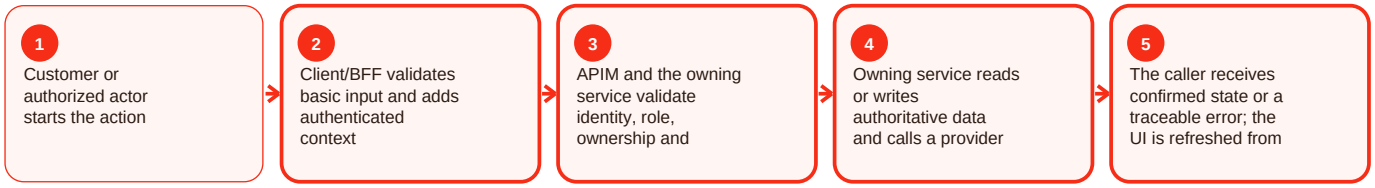
Summary

Marketplace actors is part of the platform area of Craves. Craves is a two-sided food marketplace connecting customers with home-chef supply. The repository contains a newer Next.js customer/chef web application, seven Spring Boot services, APIM guidance, legacy web/admin/API paths, infrastructure assets and milestone handovers. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The architectural direction is a front-door pattern: clients authenticate, use a same-origin Backend-for-Frontend or APIM, and reach domain services rather than databases directly. Customer and chef journeys share identity while privileged capabilities remain role-gated. For marketplace actors, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: README.md; apps/customer-web-next/README.md; services/*/README.md; infra/apim/README.md; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Marketplace actors - Functional behavior and data

Current implementation

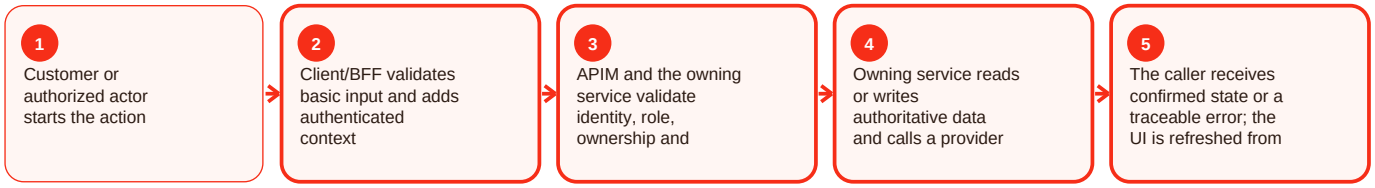
Summary

Marketplace actors is part of the platform area of Craves. Craves is a two-sided food marketplace connecting customers with home-chef supply. The repository contains a newer Next.js customer/chef web application, seven Spring Boot services, APIM guidance, legacy web/admin/API paths, infrastructure assets and milestone handovers. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The architectural direction is a front-door pattern: clients authenticate, use a same-origin Backend-for-Frontend or APIM, and reach domain services rather than databases directly. Customer and chef journeys share identity while privileged capabilities remain role-gated. For marketplace actors, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: README.md; apps/customer-web-next/README.md; services/*/README.md; infra/apim/README.md; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Marketplace actors - Operations, errors and deployment

Current implementation

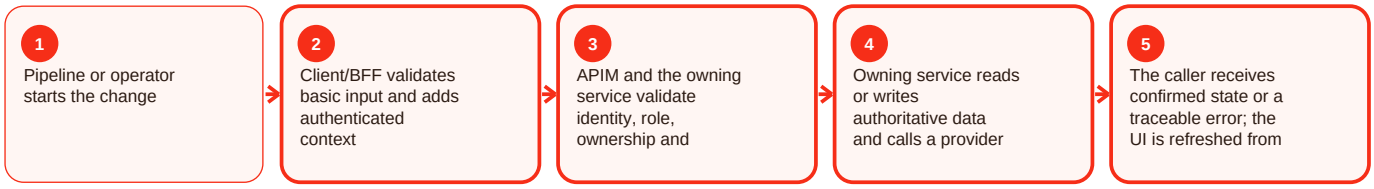
Summary

Marketplace actors is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For marketplace actors, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Customer journey - Overview and responsibility

Current implementation

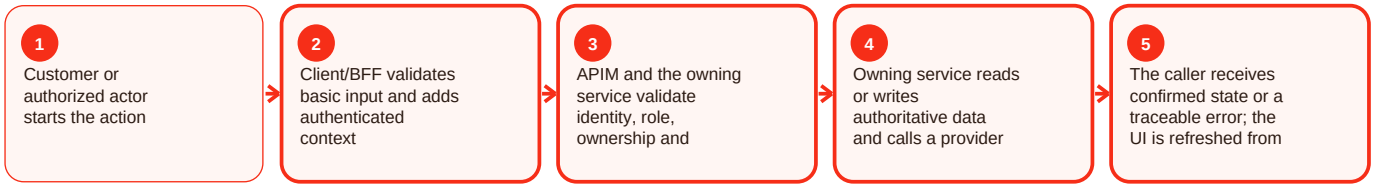
Summary

Customer journey is part of the platform area of Craves. Craves is a two-sided food marketplace connecting customers with home-chef supply. The repository contains a newer Next.js customer/chef web application, seven Spring Boot services, APIM guidance, legacy web/admin/API paths, infrastructure assets and milestone handovers. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The architectural direction is a front-door pattern: clients authenticate, use a same-origin Backend-for-Frontend or APIM, and reach domain services rather than databases directly. Customer and chef journeys share identity while privileged capabilities remain role-gated. For customer journey, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: README.md; apps/customer-web-next/README.md; services/*/README.md; infra/apim/README.md; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Customer journey - Functional behavior and data

Current implementation

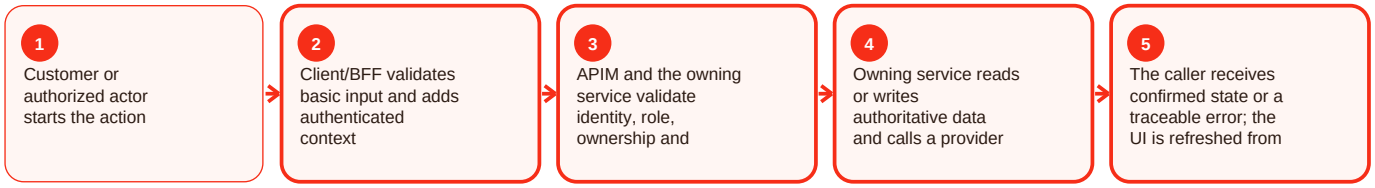
Summary

Customer journey is part of the platform area of Craves. Craves is a two-sided food marketplace connecting customers with home-chef supply. The repository contains a newer Next.js customer/chef web application, seven Spring Boot services, APIM guidance, legacy web/admin/API paths, infrastructure assets and milestone handovers. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The architectural direction is a front-door pattern: clients authenticate, use a same-origin Backend-for-Frontend or APIM, and reach domain services rather than databases directly. Customer and chef journeys share identity while privileged capabilities remain role-gated. For customer journey, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: README.md; apps/customer-web-next/README.md; services/*/README.md; infra/apim/README.md; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Customer journey - Operations, errors and deployment

Current implementation

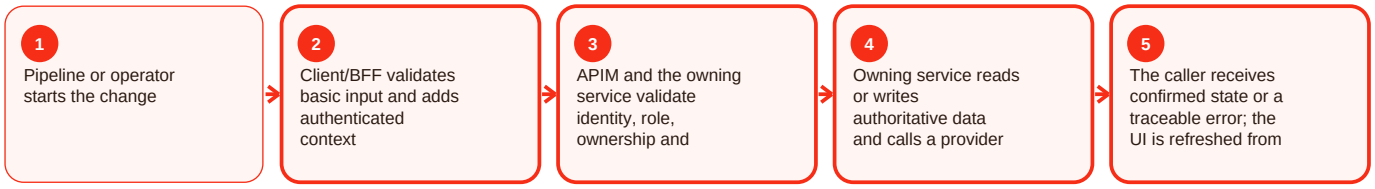
Summary

Customer journey is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For customer journey, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef journey - Overview and responsibility

Current implementation

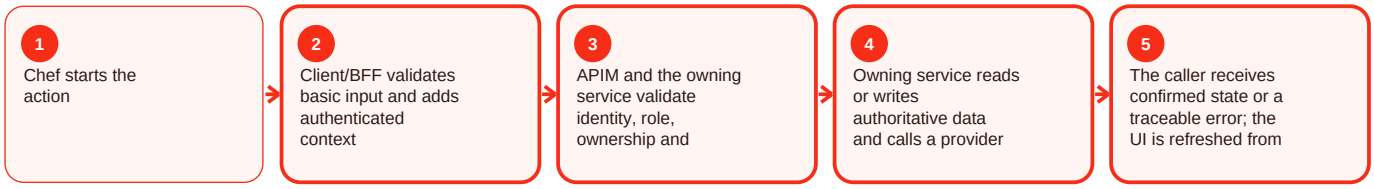
Summary

Chef journey is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef journey, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef journey - Functional behavior and data

Current implementation

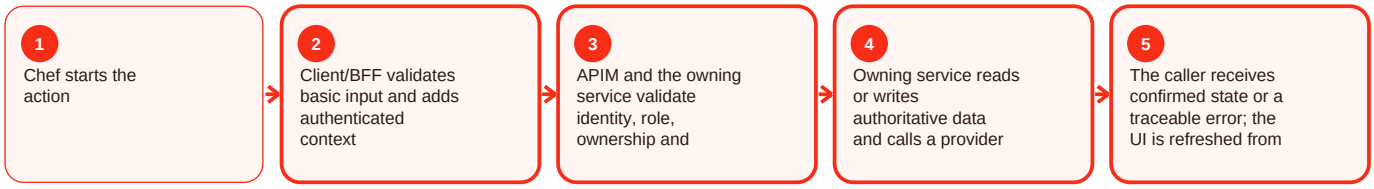
Summary

Chef journey is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef journey, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef journey - Operations, errors and deployment

Current implementation

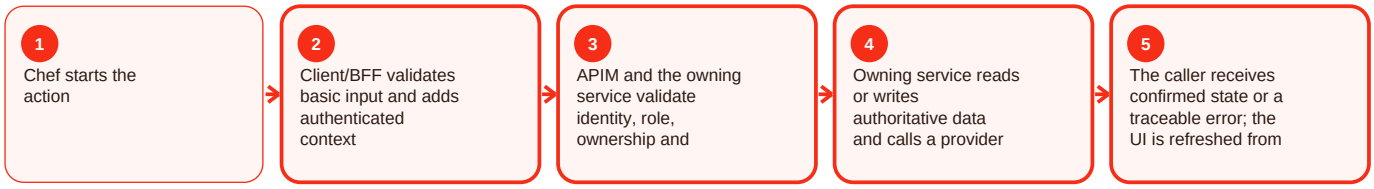
Summary

Chef journey is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef journey, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Admin / support operations - Overview and responsibility

Current implementation

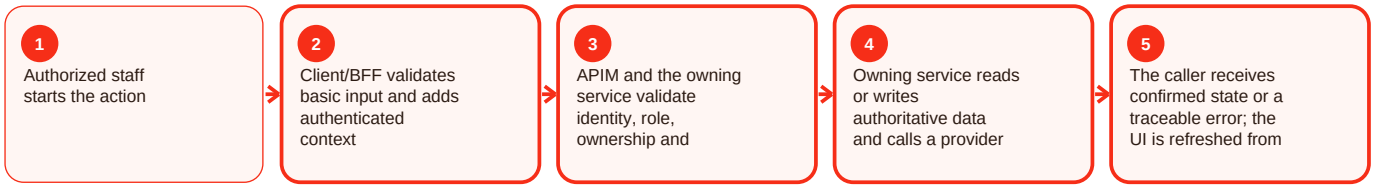
Summary

Admin / support operations is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For admin/support operations, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Admin / support operations - Functional behavior and data

Current implementation

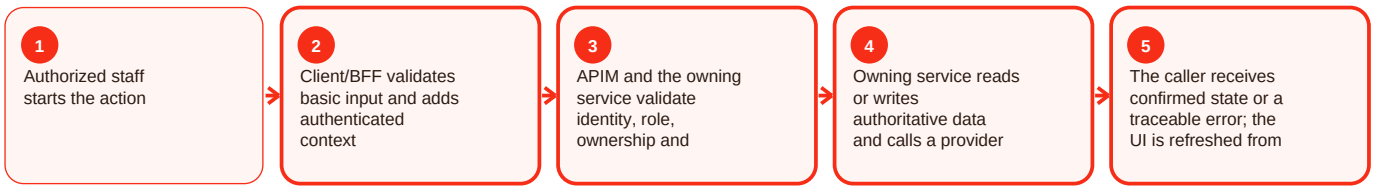
Summary

Admin / support operations is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For admin/support operations, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Admin / support operations - Operations, errors and deployment

Current implementation

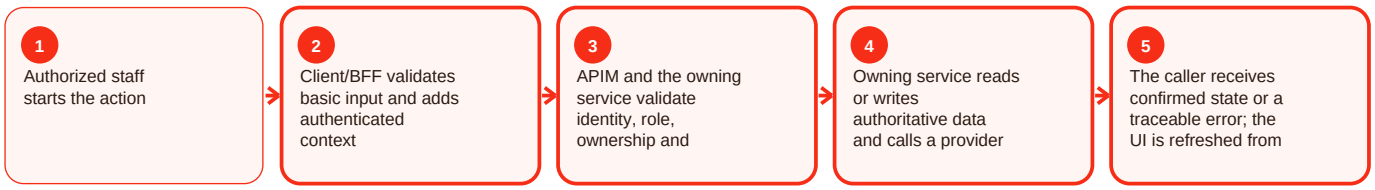
Summary

Admin / support operations is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For admin/support operations, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Discovery and catalog - Overview and responsibility

Current implementation

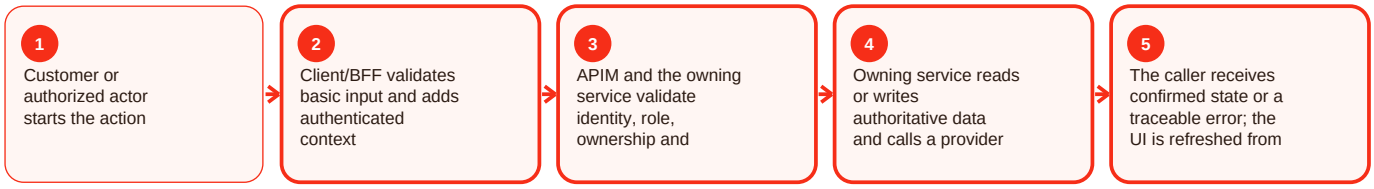
Summary

Discovery and catalog is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For discovery and catalog, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius. Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution: Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Discovery and catalog - Functional behavior and data

Current implementation

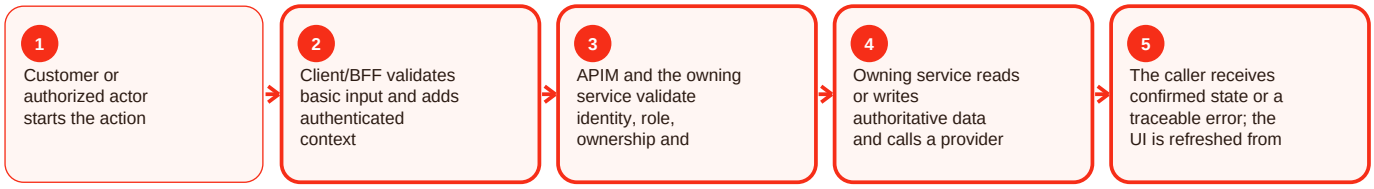
Summary

Discovery and catalog is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For discovery and catalog, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius. Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution: Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Discovery and catalog - Operations, errors and deployment

Current implementation

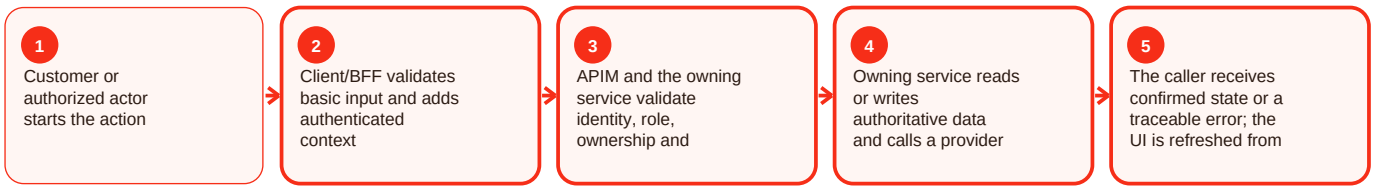
Summary

Discovery and catalog is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For discovery and catalog, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius. Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution: Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Cart and checkout - Overview and responsibility

Current implementation

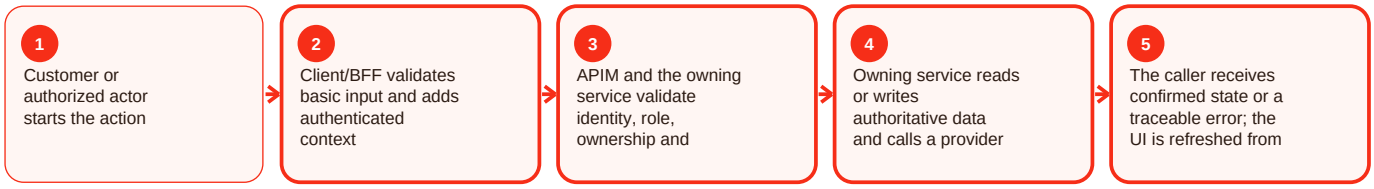
Summary

Cart and checkout is part of the checkout area of Craves. Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks. For cart and checkout, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Checkout rejected: Saved address, cart or catalog state is not valid. Resolution: Reload authoritative cart/address/catalog state and correct the failing condition.

Displayed total changed: Backend revalidated price or availability. Resolution: Show the backend quote; do not preserve a client-calculated amount.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Cart and checkout - Functional behavior and data

Current implementation

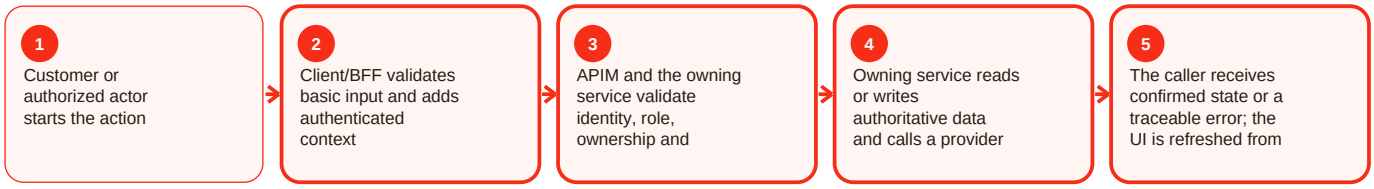
Summary

Cart and checkout is part of the checkout area of Craves. Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks. For cart and checkout, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Checkout rejected: Saved address, cart or catalog state is not valid. Resolution: Reload authoritative cart/address/catalog state and correct the failing condition.

Displayed total changed: Backend revalidated price or availability. Resolution: Show the backend quote; do not preserve a client-calculated amount.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Cart and checkout - Operations, errors and deployment

Current implementation

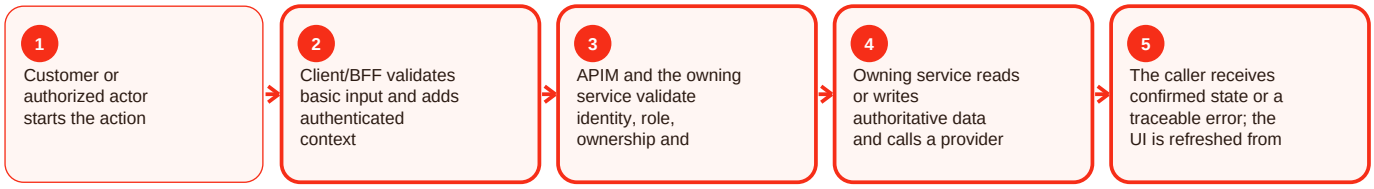
Summary

Cart and checkout is part of the checkout area of Craves. Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks. For cart and checkout, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Checkout rejected: Saved address, cart or catalog state is not valid. Resolution: Reload authoritative cart/address/catalog state and correct the failing condition.

Displayed total changed: Backend revalidated price or availability. Resolution: Show the backend quote; do not preserve a client-calculated amount.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Payments - Overview and responsibility

Current implementation

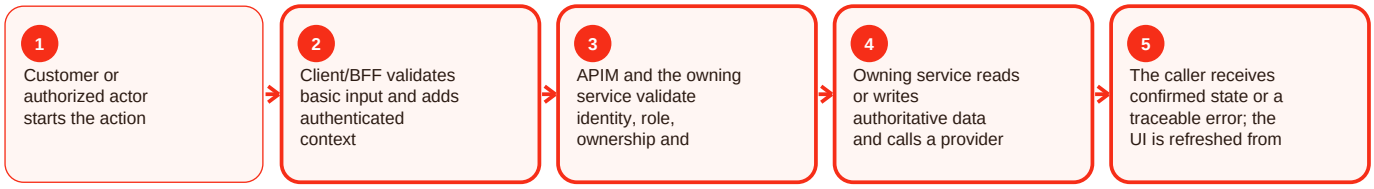
Summary

Payments is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For payments, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Payments - Functional behavior and data

Current implementation

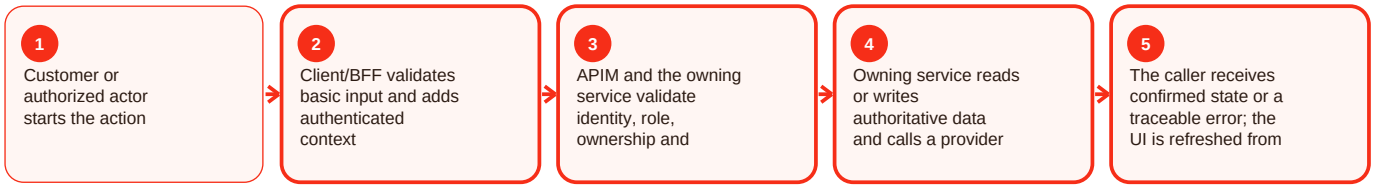
Summary

Payments is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For payments, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Payments - Operations, errors and deployment

Current implementation

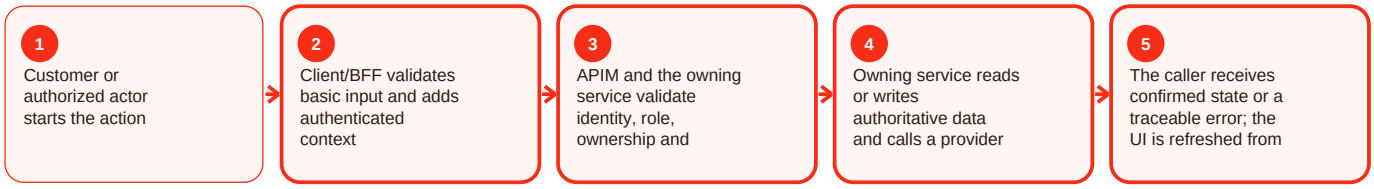
Summary

Payments is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For payments, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Orders and delivery - Overview and responsibility

Current implementation

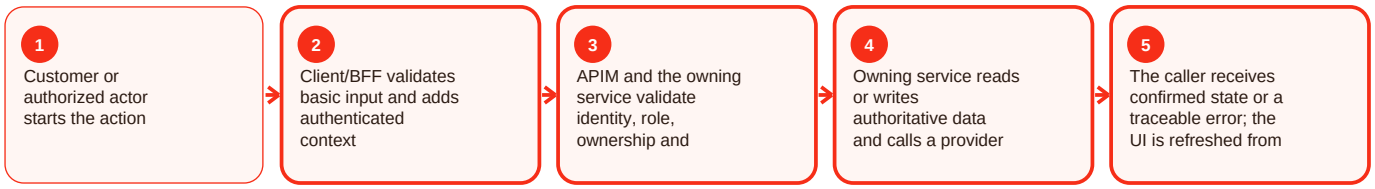
Summary

Orders and delivery is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For orders and delivery, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Order not found: Wrong ID or ownership mismatch. Resolution: Verify ID and caller context; never expose another user’s order.

State change rejected: Transition is not valid from the current state. Resolution: Reload order state and use only the supported next action.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Orders and delivery - Functional behavior and data

Current implementation

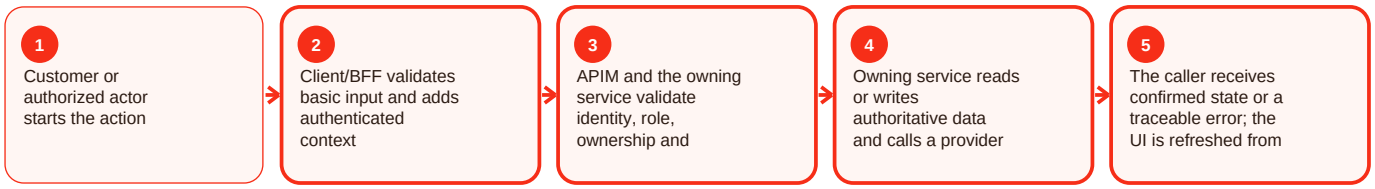
Summary

Orders and delivery is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For orders and delivery, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Order not found: Wrong ID or ownership mismatch. Resolution: Verify ID and caller context; never expose another user’s order.

State change rejected: Transition is not valid from the current state. Resolution: Reload order state and use only the supported next action.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Orders and delivery - Operations, errors and deployment

Current implementation

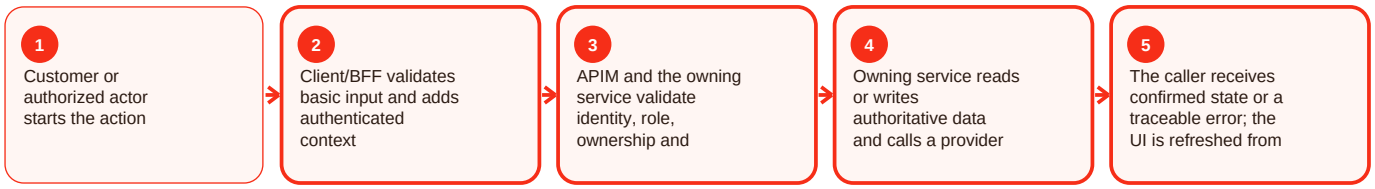
Summary

Orders and delivery is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For orders and delivery, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Order not found: Wrong ID or ownership mismatch. Resolution: Verify ID and caller context; never expose another user’s order.

State change rejected: Transition is not valid from the current state. Resolution: Reload order state and use only the supported next action.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Subscriptions and entitlements - Overview and responsibility

Current implementation

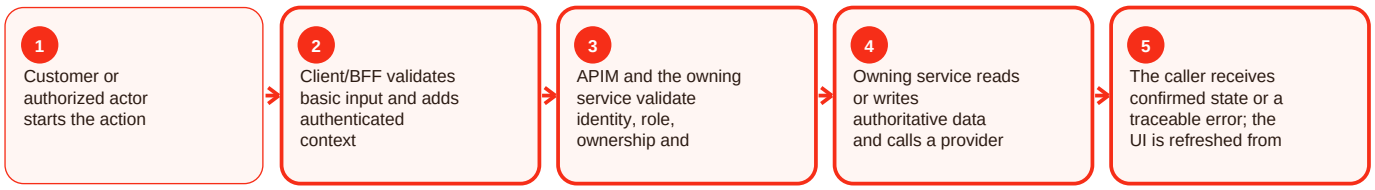
Summary

Subscriptions and entitlements is part of the subscription area of Craves. Subscriptions support repeat service and entitlements on top of one-off ordering. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

subscription-service migrations cover plans, subscriptions, payment identifiers/tokens, idempotency, request fingerprints and usage counters. The latest observed main commit adds shared entitlements and gated services. For subscriptions and entitlements, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Lifecycle action rejected: Current subscription state does not allow it. Resolution: Reload current state and use a supported transition.

Commercial rule missing: Product/Finance decision is intentionally not defined. Resolution: Document the decision before implementing it.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/subscription-service/README.md; apps/customer-web-next/src/components/plans/; apps/customer-web-next/src/components/subscriptions/; commit 3225f7fa. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Subscriptions and entitlements - Functional behavior and data

Current implementation

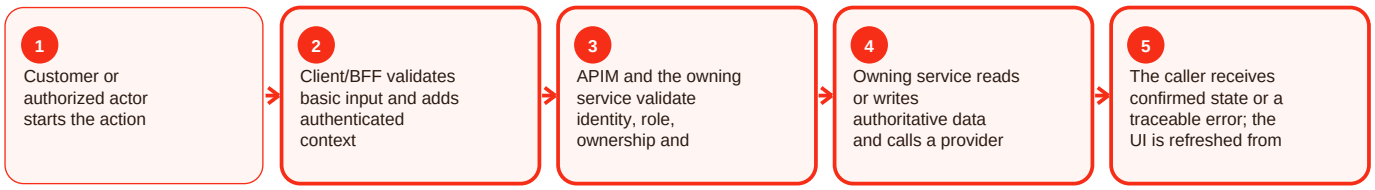
Summary

Subscriptions and entitlements is part of the subscription area of Craves. Subscriptions support repeat service and entitlements on top of one-off ordering. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

subscription-service migrations cover plans, subscriptions, payment identifiers/tokens, idempotency, request fingerprints and usage counters. The latest observed main commit adds shared entitlements and gated services. For subscriptions and entitlements, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Lifecycle action rejected: Current subscription state does not allow it. Resolution: Reload current state and use a supported transition.

Commercial rule missing: Product/Finance decision is intentionally not defined. Resolution: Document the decision before implementing it.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/subscription-service/README.md; apps/customer-web-next/src/components/plans/; apps/customer-web-next/src/components/subscriptions/; commit 3225f7fa. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Subscriptions and entitlements - Operations, errors and deployment

Current implementation

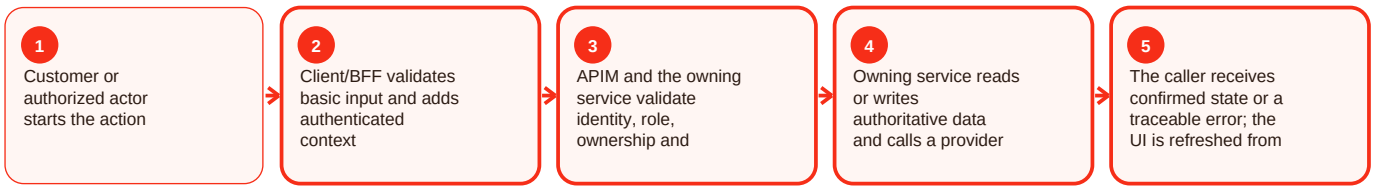
Summary

Subscriptions and entitlements is part of the subscription area of Craves. Subscriptions support repeat service and entitlements on top of one-off ordering. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

subscription-service migrations cover plans, subscriptions, payment identifiers/tokens, idempotency, request fingerprints and usage counters. The latest observed main commit adds shared entitlements and gated services. For subscriptions and entitlements, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Lifecycle action rejected: Current subscription state does not allow it. Resolution: Reload current state and use a supported transition.

Commercial rule missing: Product/Finance decision is intentionally not defined. Resolution: Document the decision before implementing it.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/subscription-service/README.md; apps/customer-web-next/src/components/plans/; apps/customer-web-next/src/components/subscriptions/; commit 3225f7fa. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Notifications - Overview and responsibility

Current implementation

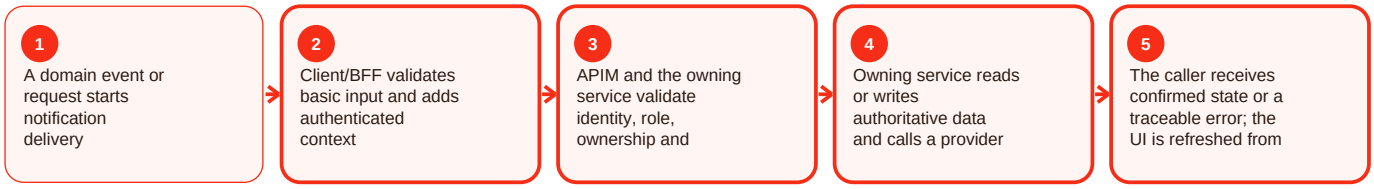
Summary

Notifications is part of the notification area of Craves. Notifications centralize email, SMS and push so domain services do not each implement vendor logic. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

notification-service documents Azure Communication Services, FCM, preferences, device tokens, optional Service Bus topic craves-domain-events, event-id idempotency, correlation propagation and a seven-day resend worker. For notifications, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

External channel stays pending: Provider adapter is disabled or unavailable. Resolution: Check channel configuration and delivery-attempt state.

Duplicate delivery: Retry did not reuse the event/request key. Resolution: Use idempotency evidence before resending.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/notification-service/README.md; apps/customer-web-next/src/components/notifications/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Notifications - Functional behavior and data

Current implementation

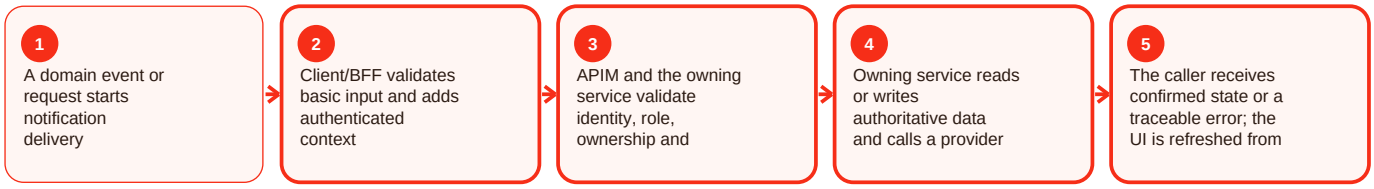
Summary

Notifications is part of the notification area of Craves. Notifications centralize email, SMS and push so domain services do not each implement vendor logic. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

notification-service documents Azure Communication Services, FCM, preferences, device tokens, optional Service Bus topic craves-domain-events, event-id idempotency, correlation propagation and a seven-day resend worker. For notifications, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

External channel stays pending: Provider adapter is disabled or unavailable. Resolution: Check channel configuration and delivery-attempt state.

Duplicate delivery: Retry did not reuse the event/request key. Resolution: Use idempotency evidence before resending.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/notification-service/README.md; apps/customer-web-next/src/components/notifications/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Notifications - Operations, errors and deployment

Current implementation

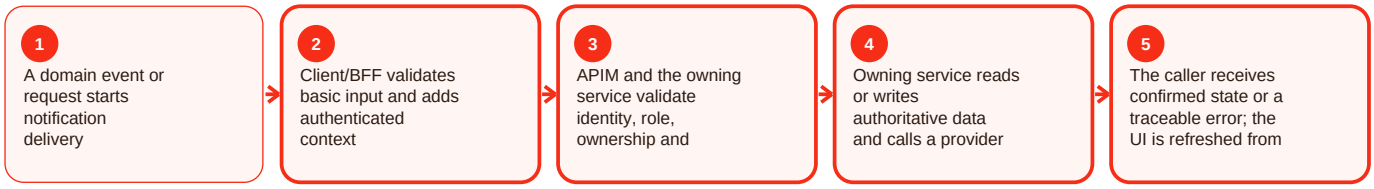
Summary

Notifications is part of the notification area of Craves. Notifications centralize email, SMS and push so domain services do not each implement vendor logic. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

notification-service documents Azure Communication Services, FCM, preferences, device tokens, optional Service Bus topic craves-domain-events, event-id idempotency, correlation propagation and a seven-day resend worker. For notifications, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

External channel stays pending: Provider adapter is disabled or unavailable. Resolution: Check channel configuration and delivery-attempt state.

Duplicate delivery: Retry did not reuse the event/request key. Resolution: Use idempotency evidence before resending.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/notification-service/README.md; apps/customer-web-next/src/components/notifications/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Location and addresses - Overview and responsibility

Current implementation

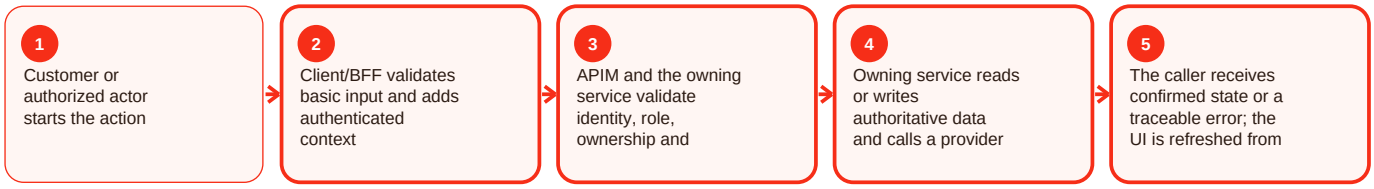
Summary

Location and addresses is part of the location area of Craves. Location/address features help a customer provide a deliverable address while retaining manual fallback when device permission or geocoding fails. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web contains location/maps/address-dialog modules. Maps are an external dependency; UI convenience does not replace backend serviceability and address validation. For location and addresses, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Current location unavailable: Permission/device/geocoder failed. Resolution: Allow manual saved-address entry and keep checkout dependent on a valid saved address.

Address rejected at checkout: Address is missing ownership/required fields/coordinates. Resolution: Fix the saved address in User-Chef before placing the order.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/src/components/location/; apps/customer-web-next/src/components/maps/; apps/customer-web-next/src/lib/location/; docs/handover/2026-08-06-customer-chef-precise-ui-address-dialog.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Location and addresses - Functional behavior and data

Current implementation

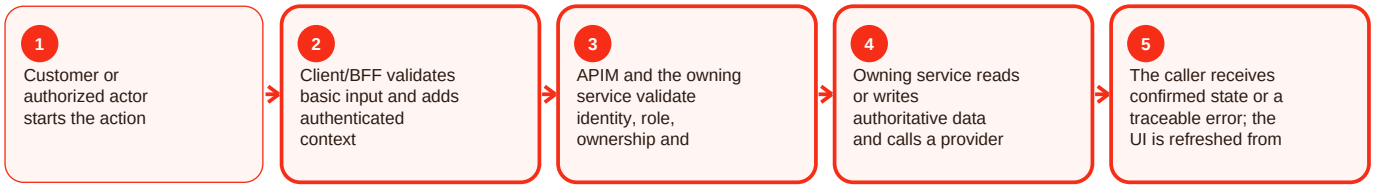
Summary

Location and addresses is part of the location area of Craves. Location/address features help a customer provide a deliverable address while retaining manual fallback when device permission or geocoding fails. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web contains location/maps/address-dialog modules. Maps are an external dependency; UI convenience does not replace backend serviceability and address validation. For location and addresses, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Current location unavailable: Permission/device/geocoder failed. Resolution: Allow manual saved-address entry and keep checkout dependent on a valid saved address.

Address rejected at checkout: Address is missing ownership/required fields/coordinates. Resolution: Fix the saved address in User-Chef before placing the order.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/src/components/location/; apps/customer-web-next/src/components/maps/; apps/customer-web-next/src/lib/location/; docs/handover/2026-08-06-customer-chef-precise-ui-address-dialog.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Location and addresses - Operations, errors and deployment

Current implementation

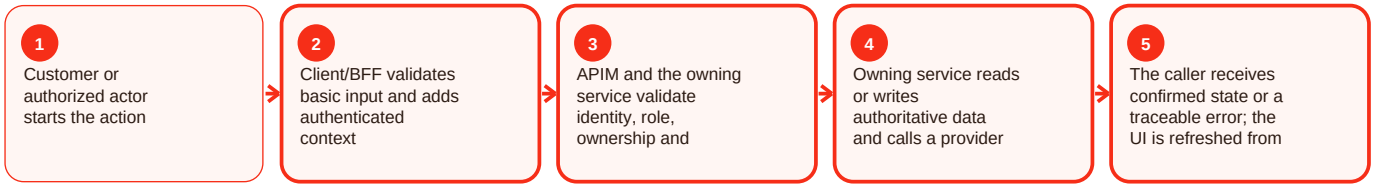
Summary

Location and addresses is part of the location area of Craves. Location/address features help a customer provide a deliverable address while retaining manual fallback when device permission or geocoding fails. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web contains location/maps/address-dialog modules. Maps are an external dependency; UI convenience does not replace backend serviceability and address validation. For location and addresses, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Current location unavailable: Permission/device/geocoder failed. Resolution: Allow manual saved-address entry and keep checkout dependent on a valid saved address.

Address rejected at checkout: Address is missing ownership/required fields/coordinates. Resolution: Fix the saved address in User-Chef before placing the order.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/src/components/location/; apps/customer-web-next/src/components/maps/; apps/customer-web-next/src/lib/location/; docs/handover/2026-08-06-customer-chef-precise-ui-address-dialog.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Consent and data rights - Overview and responsibility

Current implementation

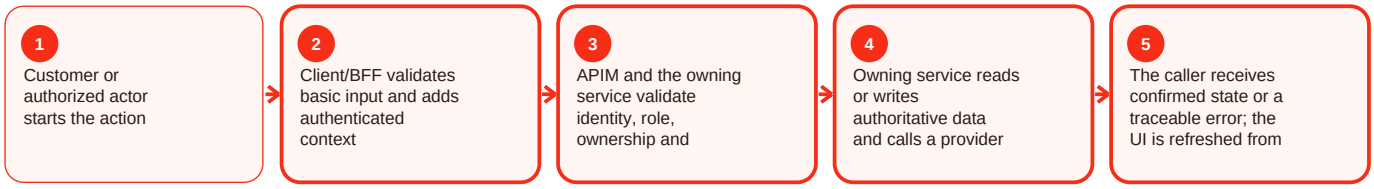
Summary

Consent and data rights is part of the privacy area of Craves. Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent. For consent and data rights, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Consent and data rights - Functional behavior and data

Current implementation

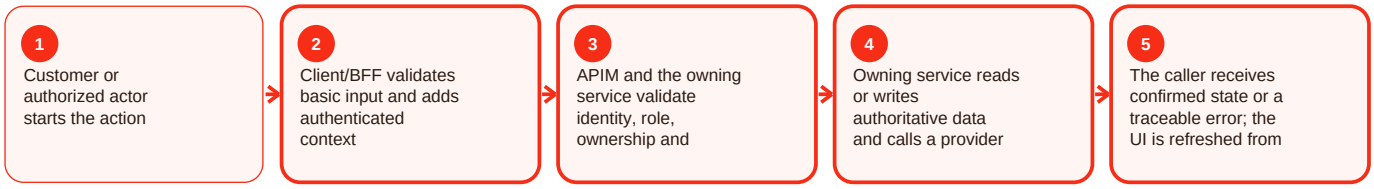
Summary

Consent and data rights is part of the privacy area of Craves. Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent. For consent and data rights, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Consent and data rights - Operations, errors and deployment

Current implementation

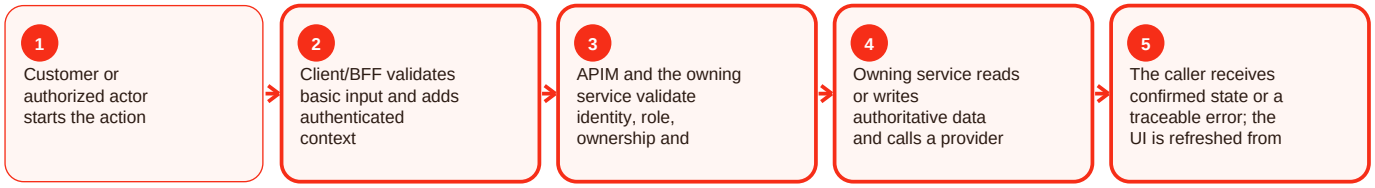
Summary

Consent and data rights is part of the privacy area of Craves. Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent. For consent and data rights, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Catering / business - Overview and responsibility

Current implementation

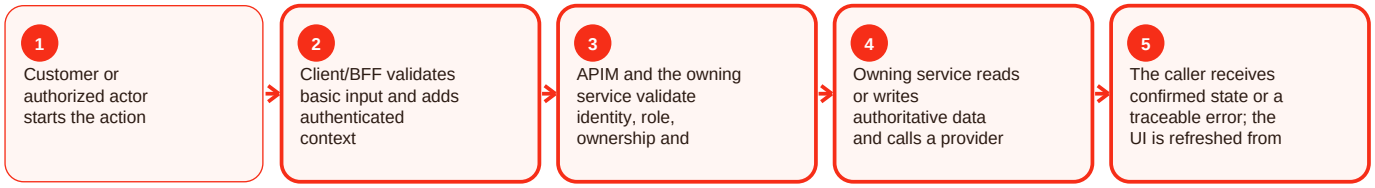
Summary

Catering / business is part of the platform area of Craves. Craves is a two-sided food marketplace connecting customers with home-chef supply. The repository contains a newer Next.js customer/chef web application, seven Spring Boot services, APIM guidance, legacy web/admin/API paths, infrastructure assets and milestone handovers. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The architectural direction is a front-door pattern: clients authenticate, use a same-origin Backend-for-Frontend or APIM, and reach domain services rather than databases directly. Customer and chef journeys share identity while privileged capabilities remain role-gated. For catering/business, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: README.md; apps/customer-web-next/README.md; services/*/README.md; infra/apim/README.md; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Catering / business - Functional behavior and data

Current implementation

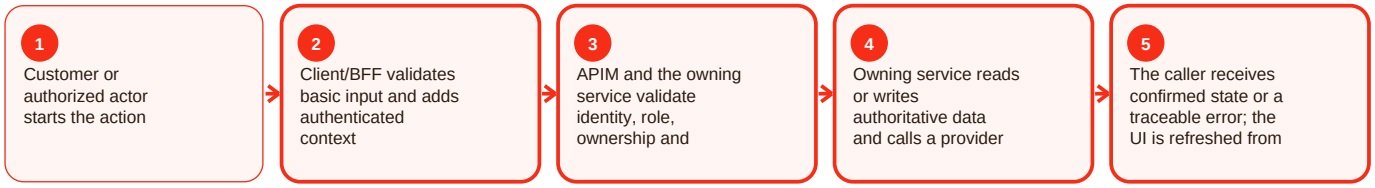
Summary

Catering / business is part of the platform area of Craves. Craves is a two-sided food marketplace connecting customers with home-chef supply. The repository contains a newer Next.js customer/chef web application, seven Spring Boot services, APIM guidance, legacy web/admin/API paths, infrastructure assets and milestone handovers. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The architectural direction is a front-door pattern: clients authenticate, use a same-origin Backend-for-Frontend or APIM, and reach domain services rather than databases directly. Customer and chef journeys share identity while privileged capabilities remain role-gated. For catering/business, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: README.md; apps/customer-web-next/README.md; services/*/README.md; infra/apim/README.md; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Catering / business - Operations, errors and deployment

Current implementation

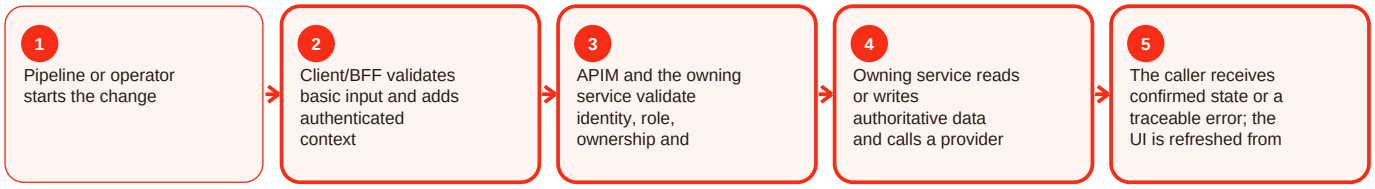
Summary

Catering / business is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For catering/business, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Ai meal concierge - Overview and responsibility

Current implementation

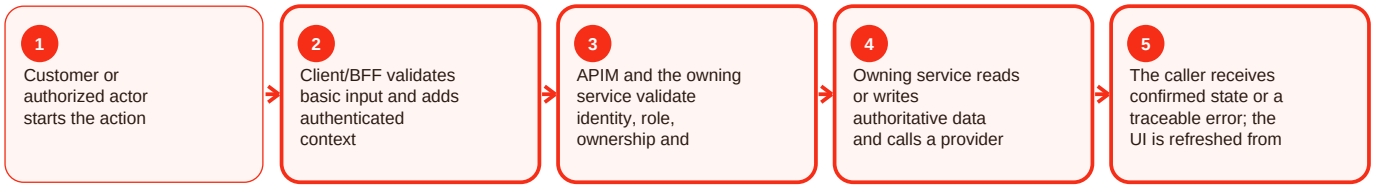
Summary

Ai meal concierge is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For AI meal concierge, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius. Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution: Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Ai meal concierge - Functional behavior and data

Current implementation

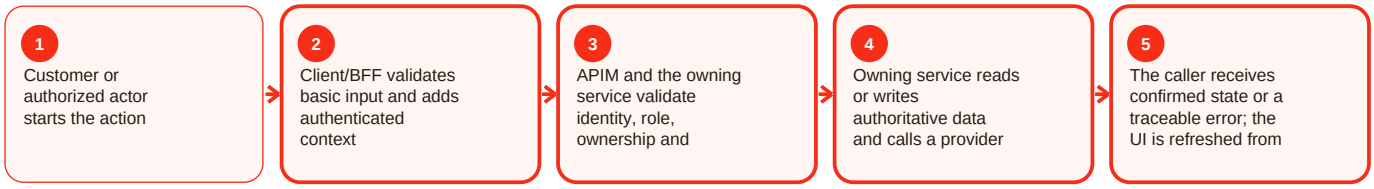
Summary

Ai meal concierge is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For AI meal concierge, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius. Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution: Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Ai meal concierge - Operations, errors and deployment

Current implementation

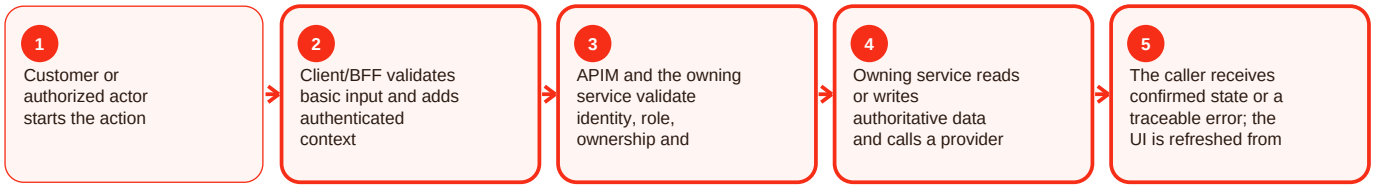
Summary

Ai meal concierge is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For AI meal concierge, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius. Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution: Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Technology platform - Overview and responsibility

Current implementation

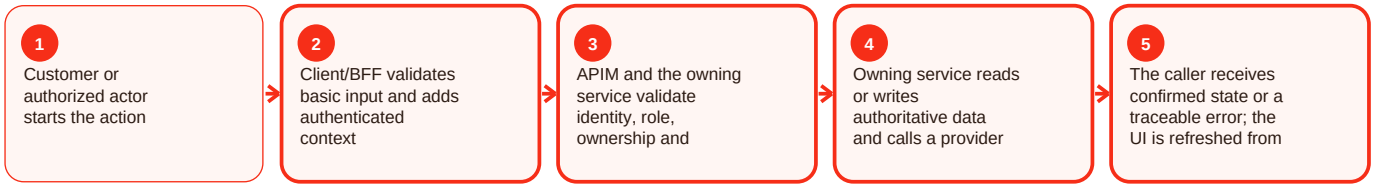
Summary

Technology platform is part of the platform area of Craves. Craves is a two-sided food marketplace connecting customers with home-chef supply. The repository contains a newer Next.js customer/chef web application, seven Spring Boot services, APIM guidance, legacy web/admin/API paths, infrastructure assets and milestone handovers. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The architectural direction is a front-door pattern: clients authenticate, use a same-origin Backend-for-Frontend or APIM, and reach domain services rather than databases directly. Customer and chef journeys share identity while privileged capabilities remain role-gated. For technology platform, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: README.md; apps/customer-web-next/README.md; services/*/README.md; infra/apim/README.md; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Technology platform - Functional behavior and data

Current implementation

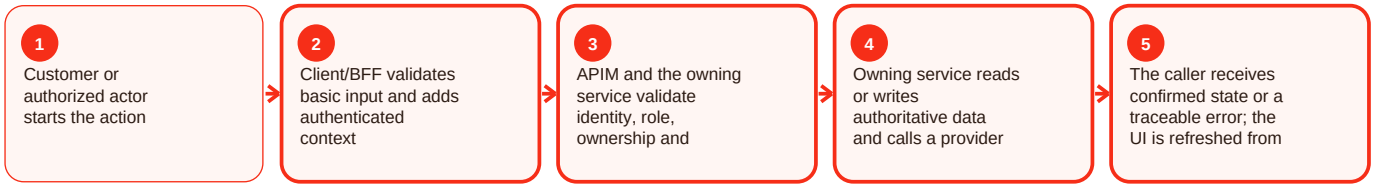
Summary

Technology platform is part of the platform area of Craves. Craves is a two-sided food marketplace connecting customers with home-chef supply. The repository contains a newer Next.js customer/chef web application, seven Spring Boot services, APIM guidance, legacy web/admin/API paths, infrastructure assets and milestone handovers. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The architectural direction is a front-door pattern: clients authenticate, use a same-origin Backend-for-Frontend or APIM, and reach domain services rather than databases directly. Customer and chef journeys share identity while privileged capabilities remain role-gated. For technology platform, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: README.md; apps/customer-web-next/README.md; services/*/README.md; infra/apim/README.md; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Technology platform - Operations, errors and deployment

Current implementation

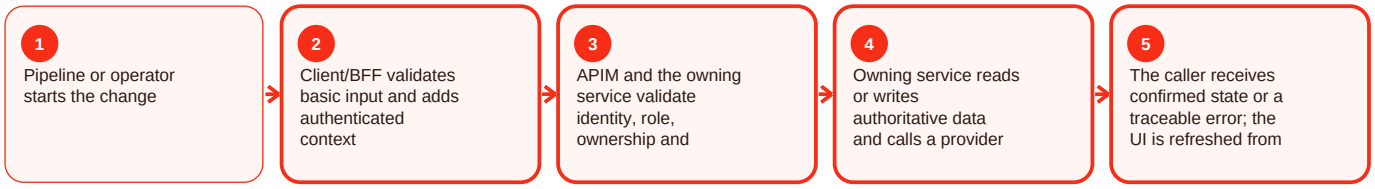
Summary

Technology platform is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For technology platform, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Legacy-to-modern transition - Overview and responsibility

Legacy / reference

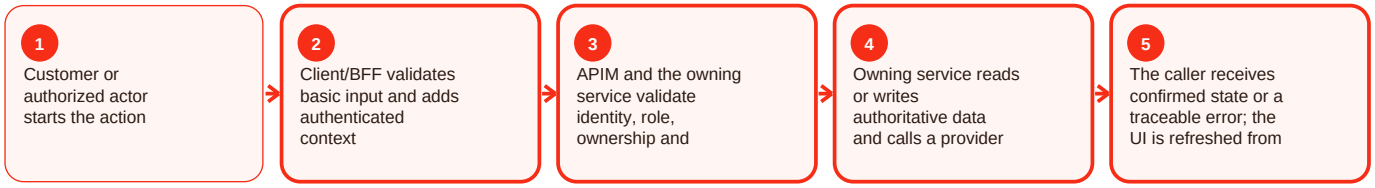
Summary

Legacy-to-modern transition is part of the legacy area of Craves. Older web/admin/Node API paths remain useful for migration history but are not automatically treated as the preferred current architecture. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The root README and apps/web/apps/admin READMEs show earlier SPA/API patterns and historical Azure preview URLs. Newer documentation centers on customer-web-next, Spring Boot services and APIM. For legacy-to-modern transition, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Useful for history and compatibility; it is not treated as the preferred current runtime. Evidence: README.md; apps/web/README.md; apps/admin/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Legacy-to-modern transition - Functional behavior and data

Legacy / reference

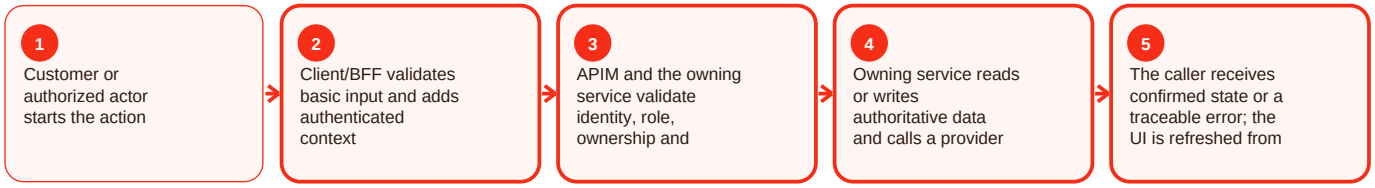
Summary

Legacy-to-modern transition is part of the legacy area of Craves. Older web/admin/Node API paths remain useful for migration history but are not automatically treated as the preferred current architecture. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The root README and apps/web/apps/admin READMEs show earlier SPA/API patterns and historical Azure preview URLs. Newer documentation centers on customer-web-next, Spring Boot services and APIM. For legacy-to-modern transition, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Useful for history and compatibility; it is not treated as the preferred current runtime. Evidence: README.md; apps/web/README.md; apps/admin/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Legacy-to-modern transition - Operations, errors and deployment

Legacy / reference

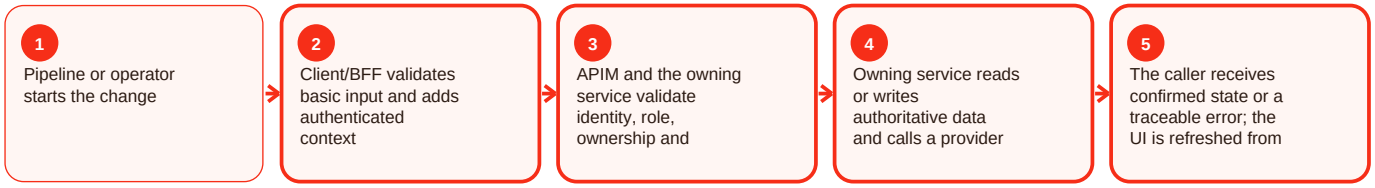
Summary

Legacy-to-modern transition is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For legacy-to-modern transition, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Useful for history and compatibility; it is not treated as the preferred current runtime. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Maturity and evidence gaps - Overview and responsibility

Needs source confirmation

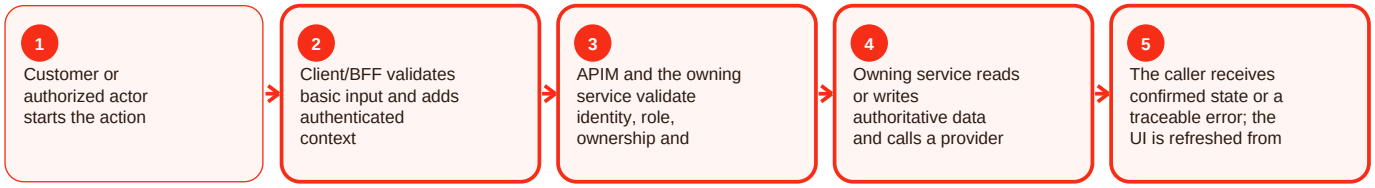
Summary

Maturity and evidence gaps is part of the platform area of Craves. Craves is a two-sided food marketplace connecting customers with home-chef supply. The repository contains a newer Next.js customer/chef web application, seven Spring Boot services, APIM guidance, legacy web/admin/API paths, infrastructure assets and milestone handovers. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The architectural direction is a front-door pattern: clients authenticate, use a same-origin Backend-for-Frontend or APIM, and reach domain services rather than databases directly. Customer and chef journeys share identity while privileged capabilities remain role-gated. For maturity and evidence gaps, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

The source reference could not be verified at the cited path, so this document does not invent the missing detail. Evidence: README.md; apps/customer-web-next/README.md; services/*/README.md; infra/apim/README.md; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Maturity and evidence gaps - Functional behavior and data

Needs source confirmation

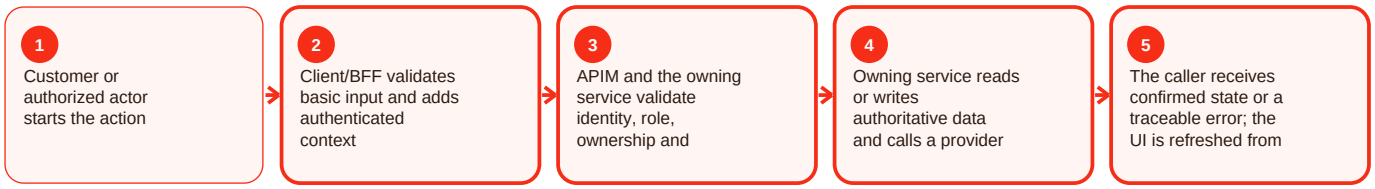
Summary

Maturity and evidence gaps is part of the platform area of Craves. Craves is a two-sided food marketplace connecting customers with home-chef supply. The repository contains a newer Next.js customer/chef web application, seven Spring Boot services, APIM guidance, legacy web/admin/API paths, infrastructure assets and milestone handovers. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The architectural direction is a front-door pattern: clients authenticate, use a same-origin Backend-for-Frontend or APIM, and reach domain services rather than databases directly. Customer and chef journeys share identity while privileged capabilities remain role-gated. For maturity and evidence gaps, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

The source reference could not be verified at the cited path, so this document does not invent the missing detail. Evidence: README.md; apps/customer-web-next/README.md; services/*/README.md; infra/apim/README.md; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Maturity and evidence gaps - Operations, errors and deployment

Needs source confirmation

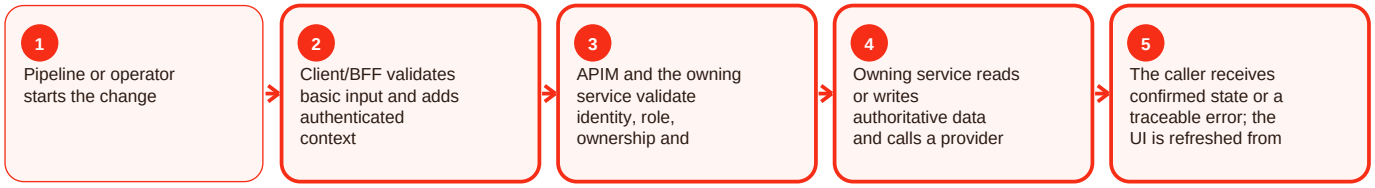
Summary

Maturity and evidence gaps is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For maturity and evidence gaps, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

The source reference could not be verified at the cited path, so this document does not invent the missing detail. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Marketplace actors - Security, evidence and change impact

Current implementation

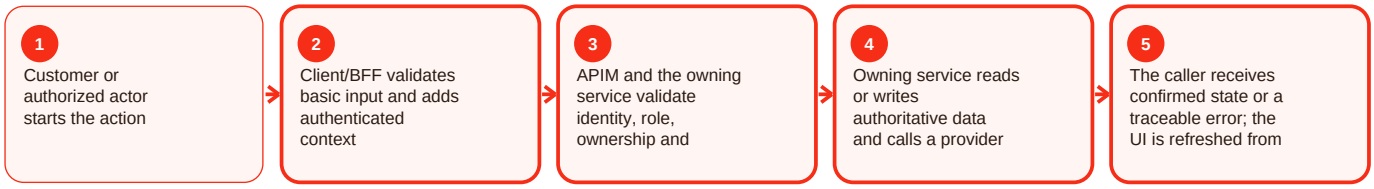
Summary

Marketplace actors is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For marketplace actors, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: infra/apim/README.md; services/user-chef-service/README.md; services/order-service/README.md; apps/customer-web-next/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Operational checklist and documented gaps

Before making a change

- Confirm the intended actor and environment.
- Confirm the current authoritative state before changing anything.
- Capture HTTP status, stable error code and request/correlation ID without secrets.
- Validate the owning component health and required dependency/configuration.
- After the action, reload the state from the backend and confirm the expected result.

Repository gaps that must stay visible

- APIM README cites infra/apim/craves-openapi.yaml, but that literal artifact was not resolvable on main during this evidence snapshot.
- customer-web-next README names azure-pipelines-customer-web-next.yml, but that literal root path was not resolvable; deployment behavior described by the README is retained while trigger/variable details are not invented.
- Native mobile has strong milestone identity/API/CI evidence, but a clearly complete runnable React Native application was not established on the reviewed main snapshot.
- Legacy preview URLs prove historical deployment references rather than current production routing.
- Commercial values such as commissions, payout percentages, subscription pricing, catering thresholds and SLAs are not fabricated where source is silent.

Definition of done

A change is complete only when the intended behavior works, authorization still fails closed, authoritative data is correct, health/readiness are good, monitoring or correlation evidence is available, the rollback path remains valid, and the documentation has been updated if the contract or operating procedure changed.